

3주차 스터디

교재 : 딥러닝 파이토치 교과서

분량 : 5.1장 ~ 5.2장

과제 기간 : ~2025-03-24

5.1 합성곱 신경망

4장에서 배운 딥러닝의 역전파를 복습해보자

- 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드로 전송된다(출력층 → 은닉층 → 입력층)
- 이러한 계산 과정은 복잡하고 많은 자원(CPU 혹은 GPU, 메모리 등)을 요구한다 + 계산 과정도 오래 걸린다
- 이 문제를 해결하고자 하는 것이 합성곱 신경망이다

합성곱 신경망은 이미지 전체를 한 번에 계산하는 것이 아닌, **이미지의 국소적 부분을 계산함**으로써 시간과 자원을 절약하여 **이미지의 세밀한 부분까지 분석할 수 있는 신경망**이다

[5.1.1 합성곱층의 필요성]

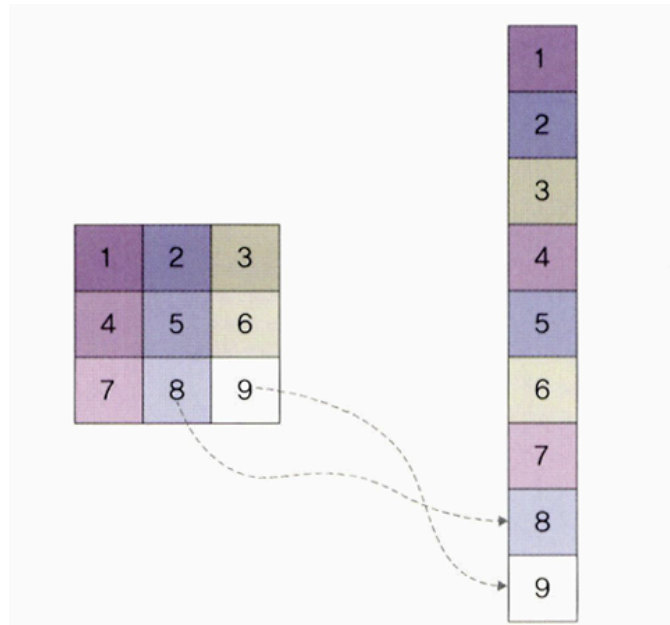
합성곱 신경망은 이미지나 영상을 처리하는 데 유용하다

ex) 3×3 흑백 이미지가 있다고 가정해보자

이미지 분석은 3x3 배열을 오른쪽과 같이 펼쳐서(flattening) 각 픽셀에 가중치를 곱하여 은닉층으로 전달한다

하지만 이미지를 펼쳐서 분석하면 데이터의 공간적 구조를 무시하게 된다

→ 이를 방지하려고 도입된 것이 합성곱층이다

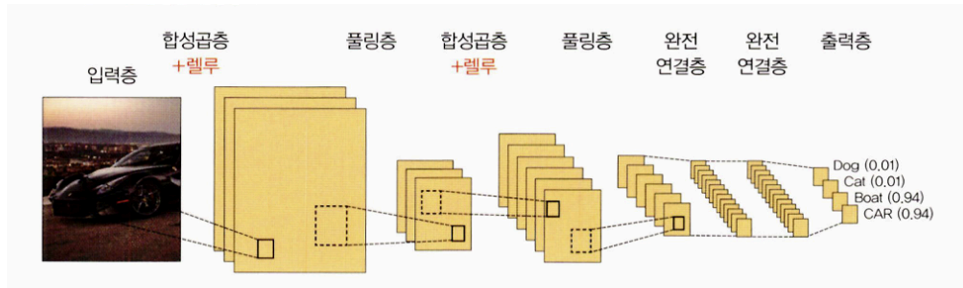


[5.1.2 합성곱 신경망 구조]

✓ 합성곱 신경망(Convolutional Neural Network, CNN)

: 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망이다

- 다차원 배열 데이터를 처리하도록 구성되어 있다
 - 컬러 이미지같은 다차원 배열 처리에 특화됨
- 다섯 개의 계층으로 구성된다
 1. 입력층
 2. 합성곱층
 3. 풀링층
 4. 완전연결층
 5. 출력층

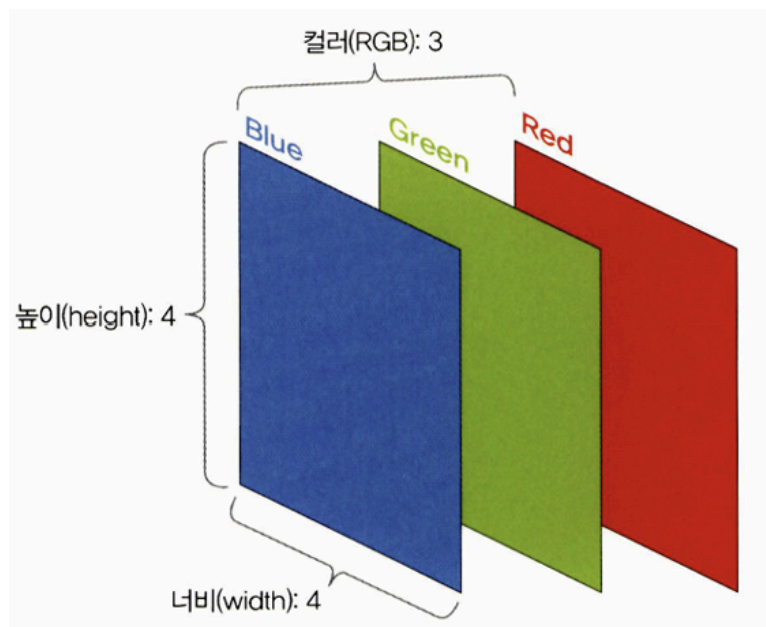


- CNN은 합성곱층과 풀링층을 거치며 **입력 이미지의 주요 특성 벡터(feature vector)**를 추출한다
- 그 후 추출된 주요 특성 벡터들은 **완전연결층**을 거치며 **1차원 벡터로 변환**된다
- 마지막 **출력층**에서 활성화 함수인 **softmax 함수**를 사용해 **최종 결과가 출력**된다

✓ 입력층(input layer)

: 입력 이미지 데일가 최초로 거치게 되는 계층

- 이미지는 높이, 너비, 채널의 값을 갖는 3차원 데이터이다
- 채널은 이미지가 그레이스케일이면 1 값, 컬러(RGB)이면 3 값을 갖는다



높이 4, 너비 4, 채널 RGB → 이미지 형태(shape)는 (4, 4, 3)

✓ 합성곱층(convolutional layer)

: 입력 데이터에서 특성을 추출하는 역할을 수행

• 특성 추출의 진행

- 입력 이미지가 들어왔을 때 이미지에 대한 특성을 감지하기 위해 커널(kernel)이나 필터를 사용한다
- 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출한다
- 추출된 결과물이 특성 맵(feature map)이다
- 커널은 3x3, 5x5 크기 적용이 일반적이다
- 스트라이드(stride)라는 지정된 간격에 따라 순차적으로 이동한다

** 스트라이드 : 필터를 적용하는 위치 간격

• 스트라이드 = 1일 때 이동하는 과정

- 1단계. 입력 이미지에 3x3 필터 적용

입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더한다



- 2단계. 필터가 1만큼 이동한다(스트라이드 = 1)



- 3단계. 필터가 1만큼 두 번째 이동한다



- 4단계. 필터가 1만큼 세 번째 이동한다



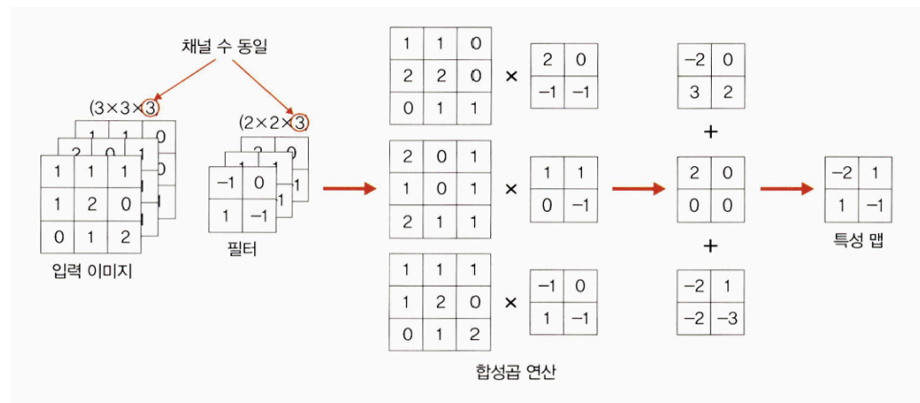
- 5단계. 필터가 1만큼 네 번째 이동한다



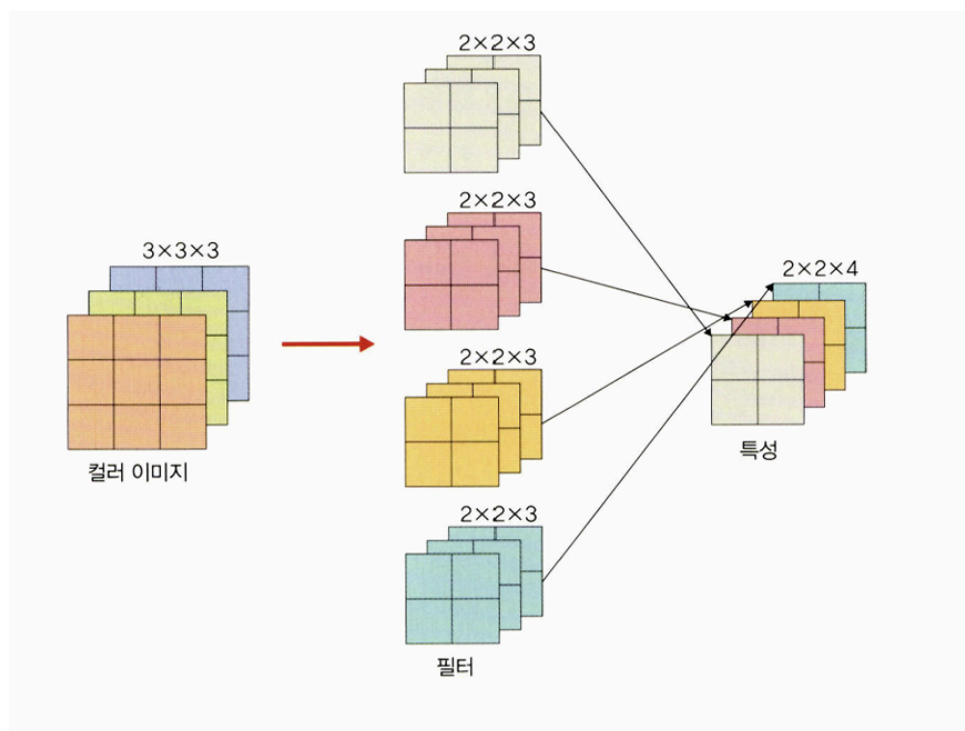
- 6단계. 필터가 1만큼 마지막으로 이동한다



- 위 과정에서 이미지 크기는 (6, 6, 1)이고 3x3 크기의 커널/필터가 스트라이드 1 간격으로 이동하면서 합성곱 연산을 수행한다
- 이렇게 커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과 합성곱 연산으로 새로운 특성 맵을 만들게 된다
- 위 과정에서는 커널과 스트라이드의 상호 작용으로 원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어들었다
- 컬러 이미지의 합성곱을 알아보자
 - 필터 채널이 3이다
 - RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해 준다
 - 필터 채널이 3이고 필터 개수는 한 개이다



- 필터가 두 개 이상인 합성곱의 형태
 - 필터가 두 개 이상이면 필터 각각은 특성 추출 결과의 채널이 된다
 - 계산은 앞의 과정과 동일하다



- 합성곱층을 요약하면 다음과 같다

- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, $\times H_1$: 세로, $\times D_1$: 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

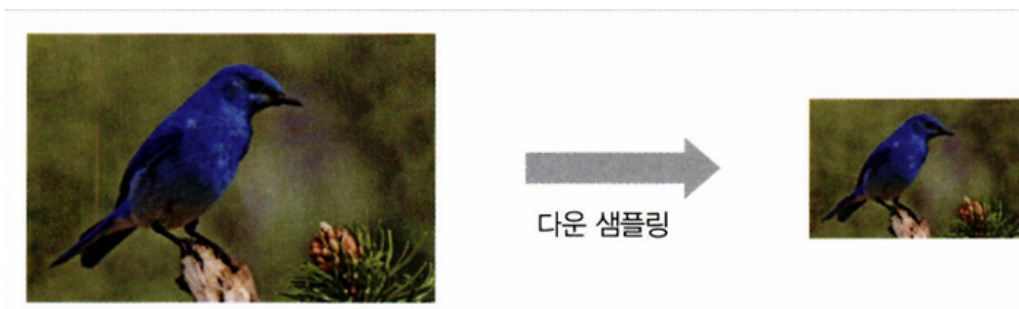
**패딩 및 CNN에 대한 추가 설명 참고 : <https://itstory1592.tistory.com/23>

✓ 풀링층(pooling layer)

: 합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 한다

+) Note : 다운 샘플링

다운 샘플링(sub-sampling)은 다음 그림과 같이 이미지를 축소하는 것이다



- 풀링 연산에는 두 가지가 사용된다

- 최대 풀링(max pooling) : 대상 영역에서 최댓값을 추출
- 평균 풀링(average pooling) : 대상 영역에서 평균을 반환
- 대부분의 CNN에서는 최대 풀링을 사용한다
 - ** 평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문이다
- 최대 풀링의 연산 과정
 - 첫 번째 최대 풀링 과정
 - 3, -1, -3, 1 값 중에서 최댓값을 선택한다



- 두 번째 최대 풀링 과정
 - 12, -1, 0, 1 값 중에서 최댓값을 선택한다



- 세 번째 최대 풀링 과정
 - 2, -3, 3, -2 값 중에서 최댓값(3)을 선택한다



- 네 번째 최대 풀링 과정
0, 1, 4, -1 값 중에서 최댓값(4)을 선택한다



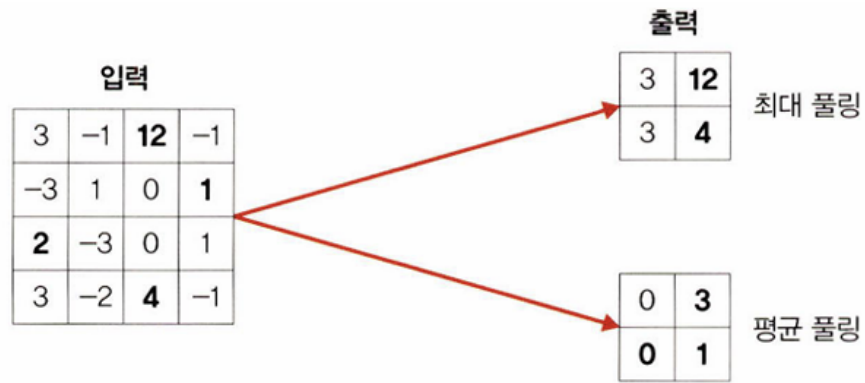
- 평균 풀링의 계산 과정은 최대 풀링과 유사한 방식으로 진행하되, 다음과 같이 각 필터의 평균으로 계산한다

$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$



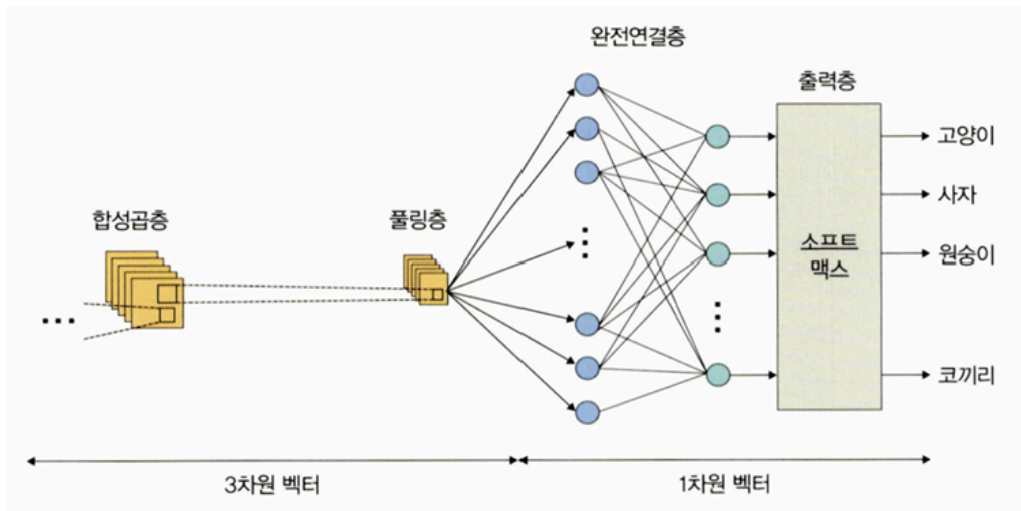
- 최대 풀링과 평균 풀링을 요약
 - 최대 풀링과 평균 풀링의 계산 과정은 다르지만 사용하는 파라미터는 동일하다

- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

✓ 완전연결층(fully connected layer)

: 합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전연결층(fully connected layer)으로 전달된다

- 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 된다



✓ 출력층(output layer)

- 출력층에서는 소프트맥스 활성화 함수가 사용되는데, 입력받은 값을 0~1 사이의 값으로 출력한다
- 마지막 출력층의 소프트맥스 함수를 사용하여 이미지가 각 레이블(label)에 속할 확률 값이 출력되며, 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정된다

[5.1.3 1D, 2D, 3D 합성곱]

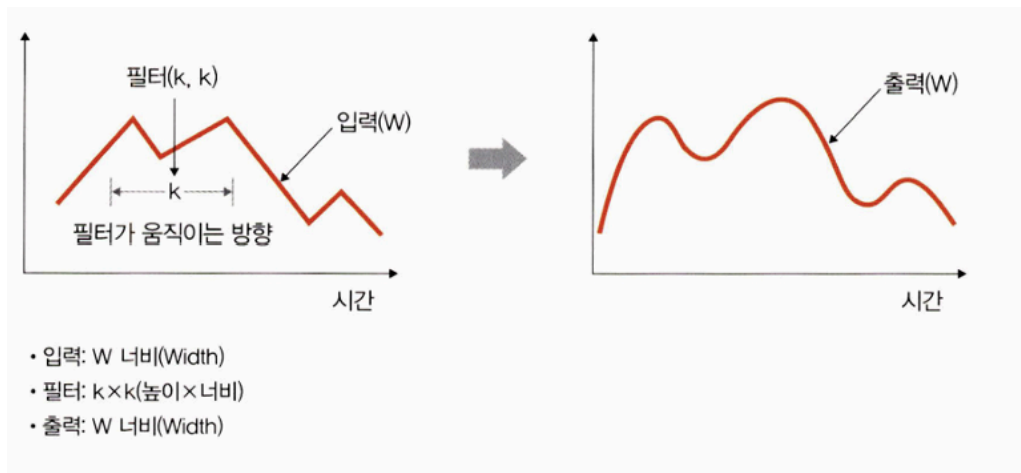
합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류할 수 있다

✓ 1D 합성곱

- 1D 합성곱은 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱이다
- 입력(W)과 필터(f)에 대한 출력은 W가 된다

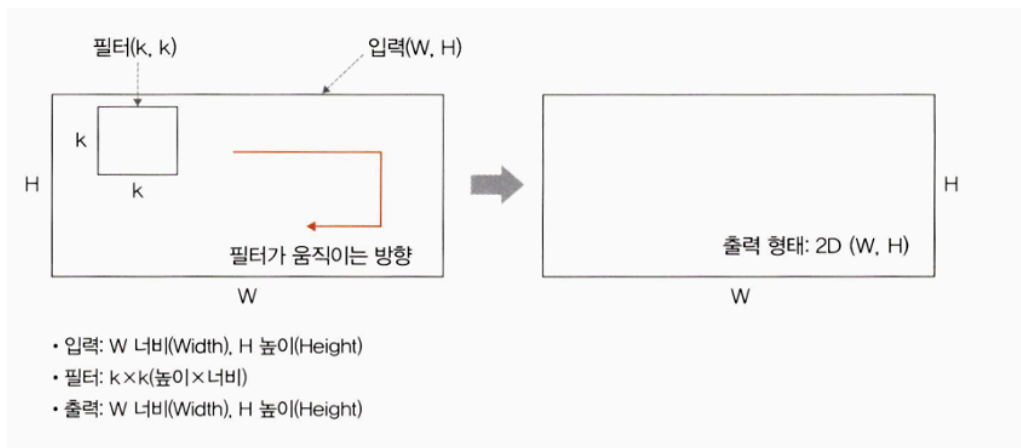
ex) 입력 [1, 1, 1, 1, 1] 필터 [0.25, 0.5, 0.25] → 출력 [1, 1, 1]

- 출력 형태는 1D 배열이 되며 그래프 곡선을 완화할 때 많이 사용된다



✓ 2D 합성곱

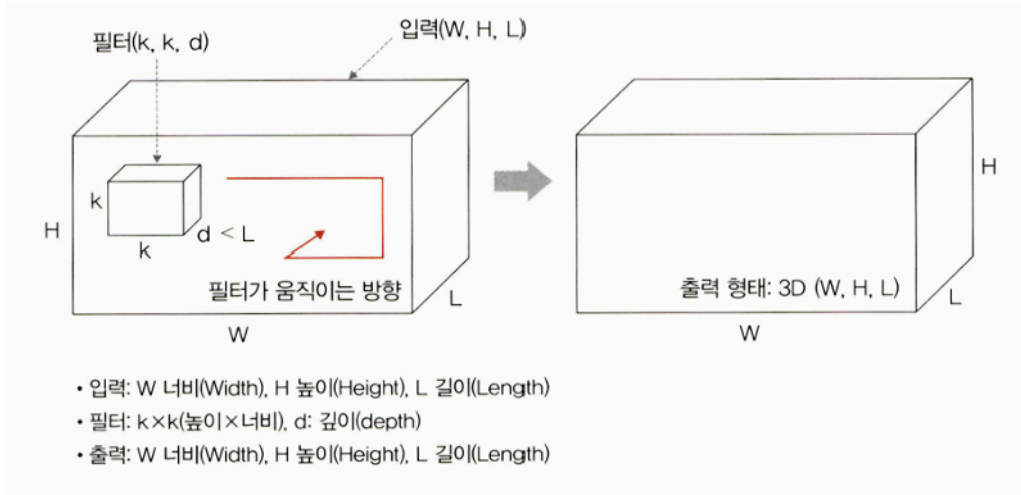
- 2D 합성곱은 필터가 두 방향으로 움직인다
- 입력 (W, H)과 필터 (k, k)에 대한 출력은 (W, H)이 되며 출력 형태는 2D 행렬이 된다



✓ 3D 합성곱

- 3D 합성곱은 필터가 세 방향으로 움직인다

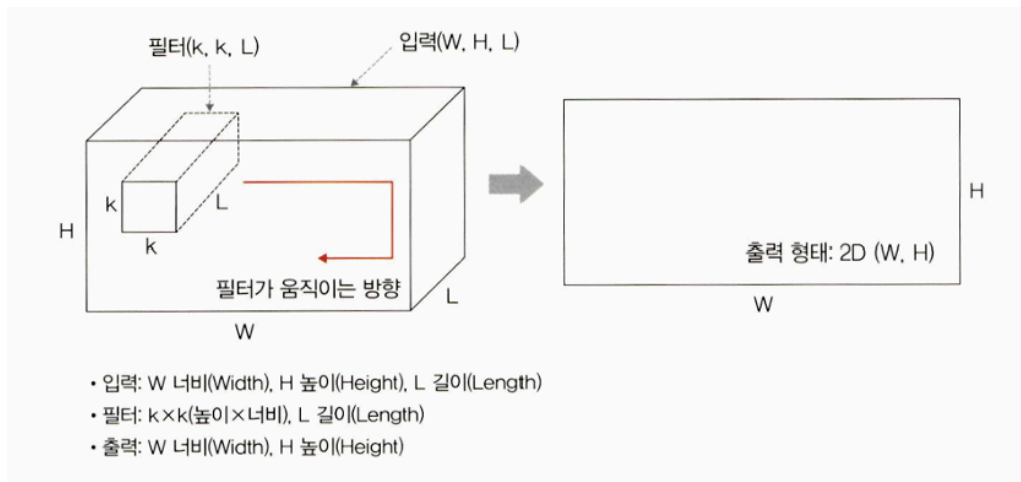
- 입력 (W, H, L)에 대해 필터 (k, k, d)를 적용하면 출력으로 (W, H, L)을 갖는 형태가 3D 합성곱이다
- 출력은 3D 형태이며, 이 때 $d < L$ 을 유지하는 것이 중요하다



✓ 3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 2D 행렬을 취한다
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문에 이와 같은 합성곱 형태가 만들어진다
- 입력(W, H, L)에 필터(k, k, L)를 적용하면 출력은 (W, H)가 된다
- 필터는 두 방향으로 움직이며 출력 형태는 2D 행렬이 된다

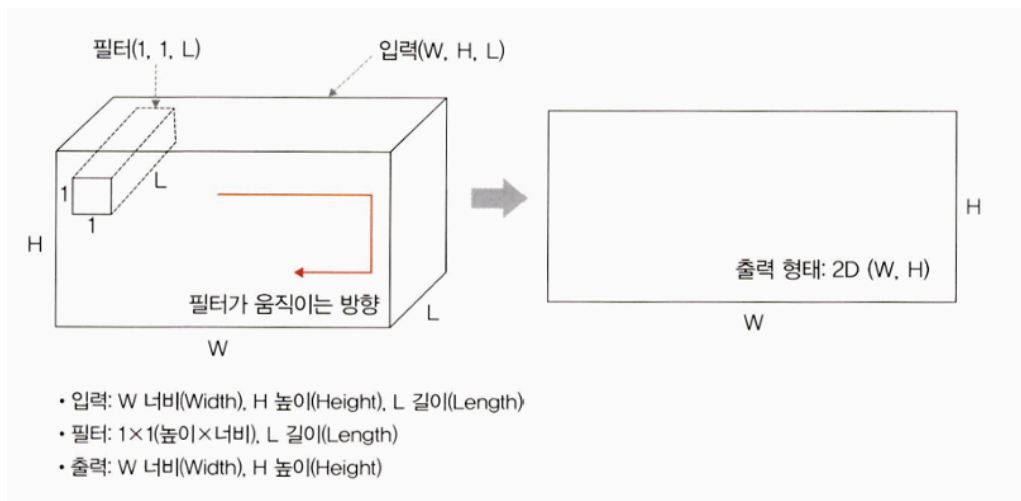
ex) LeNet-5, VGG 등이 3D 입력을 갖는 2D 합성곱의 대표적 사례이다



✓ 1x1 합성곱

- 1 × 1 합성곱은 3D 형태로 입력된다
- 입력(W, H, L)에 필터(1, 1, 1)를 적용하면 출력은 (W, H)가 된다
- 1 × 1 합성곱에서 채널 수를 조정해서 연산량이 감소되는 효과가 있다

ex) GoogLeNet



5.2 합성곱 신경망 맛보기

fashion_mnist 데이터셋을 사용해 합성곱 신경망을 직접 구현해보자

+) Note : fashion_mnist 데이터셋

- 토치비전에 내장된 예제 데이터, 운동화, 셔츠, 샌들 등의 작은 이미지 모음
- 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28x28 픽셀의 이미지 7만개로 구성되어 있음
- 훈련 데이터는 0~255 사이 값을 갖는 28x28 크기의 넘파이 배열, 레이블 데이터는 0~9까지 정수 값을 갖는 배열
- 0~9까지 정수값은 이미지의 클래스를 나타내는 레이블이며 아래와 같다

```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

🔍 합성곱 신경망 실습 예제

- 코드 5-1) 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
```



```

from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms #데이터 전처리를 위해 사용하는 리
from torch.utils.data import Dataset, DataLoader

```

- **코드 5-2) CPU 혹은 GPU 장치 확인**

```

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

```

파이토치는 기본적으로 GPU 사용을 권장한다

GPU가 장착되지 않은 환경에서도 파이토치를 정상적으로 실행*사용 가능하다

아직 교재에서는 GPU 설정을 하지 않았기에 CPU를 사용한다

+) Note : GPU 사용

일반적으로 하나의 GPU를 사용할 때는 아래와 같은 코드를 이용한다

```

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = Net()
model.to(device)

```

다수의 GPU를 사용하면 다음 코드와 같이 nn.DataParallel을 이용한다

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net()
if torch.cuda.device_count() > 1 :
    model = nn.DataParallel(model)
model.to(device)

```

- 코드 5-3) fashion_mnist 데이터셋 내려받기

```
train_dataset = torchvision.datasets.FashionMNIST("FashionMNIST/", download=True, transform=transforms.Compose([transforms.ToTensor()]))
test_dataset = torchvision.datasets.FashionMNIST("FashionMNIST/", download=True, transform=transforms.Compose([transforms.ToTensor()]))
```

(코드 설명)

torchvision.datasets는 torch.utils.data.Dataset의 하위 클래스로 다양한 데이터셋을 포함한다

torchvision.datasets에서 사용하는 주요한 파라미터는 다음과 같다

```
torchvision.datasets.FashionMNIST("../chap05/data", download = True, transform=transforms.Compose([transforms.ToTensor()]))
```

1. 첫 번째 파라미터 : FashionMNIST를 내려받을 위치를 지정한다
2. download: download를 True로 변경해 주면 첫 번째 파라미터의 위치에 해당 데이터셋이 있는지 확인한 후 내려받는다
3. transform : 이미지를 텐서(0~1)로 변경한다

- 코드 5-4) fashion_mnist 데이터를 데이터로더에 전달

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=100)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=100)
```

내려받은 fashion_mnist 데이터를 메모리로 불러오기 위해 데이터로더에 전달한다

(코드 설명)

torch.utils.data.DataLoader()를 사용하여 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록(shuffle)할 수 있다

데이터로더에서 사용하는 파라미터는 다음과 같다

```
torch.utils.data.DataLoader(train_dataset, batch_size = 100)
```

1. 첫 번째 파라미터 : 데이터를 불러올 데이터셋을 지정
2. batch_size : 데이터를 배치로 묶어준다 / 여기서는 100개 단위로 데이터를 묶어서 불러온다

• 코드 5-5) 분류에 사용될 클래스 정의

```
labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat',  
              7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'} #----- 열 개의 클래스  
  
fig = plt.figure(figsize=(8,8)); #-----출력할 이미지 가로세로 길이 단위 inch  
columns = 4;  
rows = 5;  
for i in range(1, columns*rows + 1):  
    img_xy = np.random.randint(len(train_dataset)); #----- (1)  
    img = train_dataset[img_xy][0][0,:,:] #----- (2)  
    fig.add_subplot(rows, columns, i)  
    plt.title(labels_map[train_dataset[img_xy][1]])  
    plt.axis('off')  
    plt.imshow(img, cmap='gray')  
plt.show() #----- 20개의 이미지 데이터를 시각적으로 현
```

(코드 설명)

(1) 코드 :

- np.random은 무작위로 데이터를 생성할 때 사용한다
- np.random.randint()는 이산형 분포를 갖는 데이터에서 무작위 표본을 추출할 때 사용한다
- random.randint(len(train_dataset)) 의미는 0~(train_dataset의 길이) 값을 갖는 분포에서 랜덤한 숫자 한 개를 생성하라는 의미이다

(2) 코드 :

- train_dataset을 이용한 3차원 배열을 생성
- train_dataset[img_xy][0][0,:,:) 의미는 train_dataset에서 train_dataset[img_xy][0][0,:,:)에 해당하는 요소 값을 가져오겠다는 의미이다

• 코드 5-6) 심층 신경망 모델 생성

합성곱 신경망과 심층 신경망의 비교를 위해 우선 심층 신경망 생성*학습을 진행한다

```
class FashionDNN(nn.Module):
    def __init__(self): #-----(1)
        super(FashionDNN,self).__init__()
        self.fc1 = nn.Linear(in_features=784,out_features=256) #-----(2)
        self.drop = nn.Dropout2d(0.25) #-----(3)
        self.fc2 = nn.Linear(in_features=256,out_features=128)
        self.fc3 = nn.Linear(in_features=128,out_features=10)

    def forward(self,input_data):#----- (4)
        out = input_data.view(-1, 784) #----- (5)
        out = F.relu(self.fc1(out)) #----- (6)
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

(코드 설명)

(1) 코드 :

- 클래스 형태의 모델은 항상 torch.nn.Module을 상속 받는다
- __init__() 은 객체가 갖는 속성 값을 초기화하는 역할, 객체 생성 시 자동으로 호출된다
- super(FashionDNN, self).__init__()은 FashionDNN이라는 부모 클래스를 상속받겠다는 의미로 이해하면 된다

(2) 코드 :

- nn은 딥러닝 모델(네트워크) 구성에 필요한 모듈이 모여 있는 패키지이다
- Linear 는 단순 선형 회귀 모델을 만들 때 사용한다
- `nn.Linear(in_features=784,out_features=256)` 파라미터 설명
 - in_features : 입력의 크기(input size)
 - out_features : 출력의 크기(output size)
- 실제로 데이터 연산이 진행되는 forward() 부분에는 첫 번째 파라미터 값만 넘겨주게 되고, 두 번째 파라미터에서 정의된 크기가 forward() 연산의 결과가 된다

(3) 코드 :

- torch.nn.Dropout(p)는 p만큼 비율로 텐서의 값이 0이 되고, 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커진다
- ex) p=0.3의 의미는 전체 값 중 0.3의 확률로 0이 된다는 뜻, 0이 되지 않는 0.7에 해당하는 값은 $(1/(1-0.7))$ 만큼 커진다

(4) 코드 :

- forward() 함수는 모델이 학습 데이터를 입력받아 순전파 학습을 진행시키며, 반드시 forward라는 이름의 함수여야한다
- forward()는 모델이 학습 데이터를 입력 받아서 순전파 연산을 진행하는 함수이다
- 객체를 데이터와 함께 호출하면 자동으로 실행된다
- 순전파 연산이란 $H(x)$ 식에 입력 x로부터 예측된 y를 얻는 것이다

(5) 코드 :

- 파이토치에서 사용하는 view는 넘파이의 reshape과 같은 역할이다
- 텐서의 크기를 변경해주는 역할이다
- `input_data.view(-1, 784)` 는 `input_data.view(?, 784)`의 크기로 변경하라는 의미이다
- 첫 번째 차원 -1은 사용자가 잘 모르겠으니 파이토치에 맡기겠다는 의미이고 두 번째 차원의 길이는 784를 갖도록 하라는 의미이다
- 2차원의 텐서로 변경하되 (?, 784)의 크기로 변경하라는 의미이다

(6) 코드 :

- 활성화 함수는 다음 두 가지 방법으로 지정이 가능하다
 - `F.relu()` : `forward ()` 함수에서 정의
 - `nn.ReLU()` : `__init__()` 함수에서 정의
- 활성화 함수 사용에 한정하여 이 둘 간의 차이는 간단히 사용하는 위치라고 할 수 있다
- BUT 근본적으로는 `nn.functional.xx()`와 `nn.xx()`는 사용 방법에 차이가 있다

구분	<code>nn.xx</code>	<code>nn.functional.xx</code>
형태	<code>nn.Conv2d</code> : 클래스 <code>nn.Module</code> 클래스를 상속받아 사용	<code>nn.functional.conv2d</code> : 함수 <code>def function (input)</code> 으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	<code>nn.Sequential</code> 내에 위치	<code>nn.Sequential</code> 에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

• 코드 5-7) 심층 신경망에서 필요한 파라미터 정의

```
learning_rate = 0.001;
model = FashionDNN();
model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate); #-----
print(model)
```

모델을 학습시키기 전에 손실 함수, 학습률, 옵티마이저에 대해 정의해준다

(코드 설명)

(1) 코드 :

- 옵티마이저를 위한 경사 하강법은 Adam을 사용한다
- 학습률을 의미하는 lr은 0.001을 사용한다

- 이 코드를 실행하면 심층 신경망 모델을 보여준다

• 코드 5-8) 심층 신경망을 이용한 모델 학습

```

num_epochs = 5
count = 0
loss_list = [] #------(1)
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader: #------(2)
        images, labels = images.to(device), labels.to(device) #------(3)

        train = Variable(images.view(100, 1, 28, 28)) #------(4)
        labels = Variable(labels)

        outputs = model(train) #-----학습 데이터를 모델에 용
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50) : #-----count를 50으로 나눴을 때 나머지가 0이 아냐
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)

```

```

        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)

    accuracy = correct * 100 / total #----- (5)
    loss_list.append(loss.data) #----- (1)'
    iteration_list.append(count)
    accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data

```

심층 신경망에 데이터를 적용해 모델을 학습시킨다

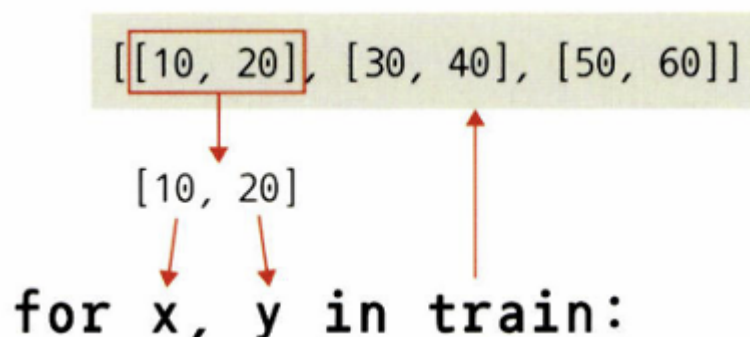
(코드 설명)

(1) 코드 :

- 일반적으로 배열이나 행렬과 같은 리스트를 사용하는 방법은 아래와 같다
 - (1)과 같이 비어 있는 배열이나 행렬을 만든다
 - (1)' 처럼 append 메서드를 이용해 데이터를 하나씩 추가한다

(2) 코드 :

- for 구문을 사용해 레코드를 하나씩 가져온다
- for x, y in train : 과 같이 in 앞에 변수를 두 개 지정해 주면 레코드에서 요소 두 개를 꺼내 오겠다는 의미이다



(3) 코드 :

- 모델이 데이터를 처리하기 위해서는 모델과 데이터가 동일한 장치에 있어야한다 (CPU 또는 GPU)
- 코드 5-7 model.to(device)가 GPU를 사용했다면 images.to(device), labels.to(device)도 GPU에서 처리되어야 한다
- CPU에서 처리된 데이터를 GPU 모델에 적용하거나 그 반대의 경우 런타임 오류가 발생한다

(4) 코드 :

- Autograd는 자동 미분을 수행하는 파이토치의 핵심 패키지
- 자동 미분에 대한 값을 저장하기 위해 tape를 사용한다
- 순전파 단계에서 테이프는 수행하는 모든 연산을 저장한다
- 역전파 단계에서 저장된 값들을 꺼내서 사용한다

⇒ Autograd는 Variable을 사용해서 역전파를 위한 미분 값을 자동으로 계산해 준다

⇒ 자동 미분을 계산하기 위해서는 torch.autograd 패키지 안에 있는 Variable을 이용해야 동작한다

(5) 코드 :

- 분류 문제에 대한 정확도는 전체 예측에 대한 정확한 예측의 비율로 표현할 수 있다
- 코드는 다음과 같다

```
classification accuracy = correct predictions / total predictions
```

```
classification accuracy = correct predictions / total predictions * 100
```

- 분류 문제에 대한 정확도는 다음과 같이 값을 반전시켜 오분류율 또는 오류율로 표현할 수 있다

```
error rate = ( 1 - (correct predictions / total predictions)) * 100
```

- 코드 실행 결과

아래 처럼 모델 훈련 결과가 출력되며 정확도가 높은 수치(86%)를 보인다

```

Iteration: 500, Loss: 0.5627565979957581, Accuracy: 82.70999908447266%
Iteration: 1000, Loss: 0.4662465751171112, Accuracy: 84.4000015258789%
Iteration: 1500, Loss: 0.33023178577423096, Accuracy: 84.41000366210938%
Iteration: 2000, Loss: 0.3131774067878723, Accuracy: 85.05999755859375%
Iteration: 2500, Loss: 0.28404906392097473, Accuracy: 86.26000213623047%
Iteration: 3000, Loss: 0.31252551078796387, Accuracy: 86.4800033569336%

```

- 코드 5-9) 합성곱 네트워크 생성

```

class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential( #----- (1)
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), #----- (3)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2) #----- (4)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(in_features=64*6*6, out_features=600) #----- (5)
        self.drop = nn.Dropout2d(0.25)
        self.fc2 = nn.Linear(in_features=600, out_features=120)
        self.fc3 = nn.Linear(in_features=120, out_features=10) #----- 마지막

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1) #----- (6)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

(코드 설명)

(1) 코드 :

- `nn.Sequential` 을 사용하면 `__init__()`에서 사용할 네트워크 모델을 정의해준다
- 또한 `forward()` 함수에서 구현될 순전파를 계층 형태로 좀 더 가독성이 뛰어난 코드로 작성할 수 있다

⇒ 즉 계층을 차례로 쌓을 수 있도록 $Wx+b$ 와 같은 수식과 활성화 함수를 연결해주는 역할을 한다

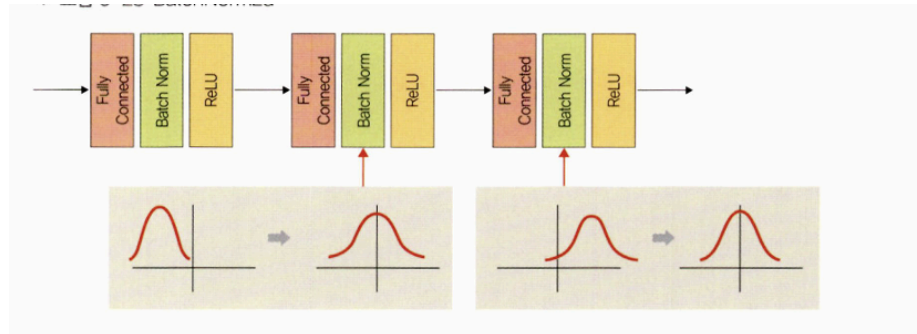
- 특히 데이터가 각 계층을 순차적으로 지나갈 때 사용하면 좋은 방법이다
- `nn.Sequential`은 여러 개의 계층을 하나의 컨테이너에 구현하는 방법 이다

(2) 코드 :

- 합성곱층은 합성곱 연산을 통해 이미지 특징을 추출한다
- 합성곱이란 커널이라는 $n \times m$ 크기의 행렬을 높이 x 너비 크기의 이미지를 처음부터 끝까지 훑으며 각 원소 값끼리 곱한 후 모두 더한 값을 출력한다
- 커널의 사이즈는 일반적으로 3×3 , 5×5 이다
- `nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)` 의 파라미터 설명은 아래와 같다
 - `in_channels` : 입력 채널 수
 - `out_channels` : 출력 채널 수
 - `kernel_size` : 커널 크기(=필터)
 - 커널은 이미지 특징을 찾아내기 위한 공용 파라미터이며 CNN에서 학습 대상은 필터 파라미터가 된다
 - 커널은 입력 데이터를 스트라이드 간격으로 순회하며 합성곱을 계산한다
 - `kernel_size = 3` 일 때 커널 크기는 $(3,3)$ 의 정사각형을 의미하며 직사각형을 사용하고 싶으면 $(3,5)$ 처럼 따로 지정해야다
 - `padding` : 패딩 크기
 - 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채운다
 - 패딩 값이 클수록 출력 크기도 커진다

(3) 코드 :

- BatchNorm2d는 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 갖더라도 평균과 분산을 이용해 정규화하는 것을 의미한다
- 아래 그림처럼 배치 단위나 계층에 따라 입력 값의 분포가 모두 다르더라도 정규화를 통해 가우시안 분포 형태로 만든다
→평균은 0, 표준편차는 1로 데이터 분포가 조정된다



(4) 코드 :

- MacPool2d는 이미지 크기를 축소시키는 용도이다
- 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용한다
- 풀링 계층의 처리 방법은 최대 풀링과 평균 풀링이 있다
- `nn.MaxPool2d(kernel_size=2, stride=2)` 파라미터 설명
 - kernel_size : m x n 행렬로 구성된 가중치
 - stride : 입력 데이터에 커널을 적용할 때 이동할 간격, 스트라이드 값이 커지면 출력 크기는 작아다

(5) 코드 :

- 클래스를 분류하기 위해서 이미지 형태의 데이터를 배열 형태로 변환해 작업해야 한다
- 이 때 Conv2d에서 사용하는 하이퍼파라미터 값들에 따라서 출력 크기(output size)가 달라진다

⇒ 패딩과 스트라이드의 값에 따라서 출력 크기가 달라진다

- 줄어든 출력 크기는 완전연결층(fully connected layer, 최종적으로 분류를 담당)으로 전달된다
- `nn.Linear(in_features=64*6*6, out_features=600)` 의 파라미터 설명

- in_features : 입력 데이터의 크기
- out_features : 출력 데이터의 크기

(6) 코드 :

- 합성곱층에서 완전연결층으로 변경되므로 데이터 형태를 1차원으로 바꿔주어야 한다
- out.size(0)은 결국 100을 의미한다

⇒ (100,?) 크기의 텐서로 변경하겠다는 의미이다

- out.view(out.size(0), -1)에서 -1은 행의 수는 정확히 알고 있지만 열의 수를 알지 못할 때 사용한다

• 코드 5-10) 합성곱 네트워크를 위한 파라미터 정의

```
learning_rate = 0.001;
model = FashionCNN();
model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)
```

결과로는 합성곱 네트워크의 구조에 대해 출력된다

• 코드 5-11) 합성곱 네트워크를 위한 파라미터 정의

5-8 코드와 동일하게 학습 데이터를 이용하여 모델을 학습시킨다

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []
```

```

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.d

```

- 결과로는 모델 훈련 결과가 출력된다

- 심층 신경망보다 정확도가 약간 높다
- 이미지 데이터가 많아지면 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능하므로 합성곱 신경망을 생성할 수 있도록 학습하는게 중요하다